

ATELIER 3

Spring Data JPA : interfaces repository et opérations CRUD

Support technique : Java 17+ / Maven / IntelliJ IDEA / Spring Boot / Spring Data JPA / Hibernate / Lombok / SonarQube for IDE

Objectifs pédagogiques

- Comprendre le rôle de **Spring Data JPA** dans la gestion de la couche de persistance d'une application Spring Boot.
- Identifier l'architecture de Spring Data JPA et expliquer le rôle de chaque interface : **Repository**, **CrudRepository**, **ListCrudRepository**, **PagingAndSortingRepository** et **JpaRepository**.
- Mettre en place la **couche Repository** dans le package **repository**, en nommant chaque interface avec le préfixe **I** (ex. **IVehiculeRepository**).
- Réaliser les opérations **CRUD** avec **CrudRepository** et comprendre le comportement exact de **save()**, **findById()** et **deleteById()**.
- Choisir l'interface adaptée à un besoin donné et appliquer les bonnes pratiques de la couche Repository.
- Utiliser **SonarQube for IDE (SonarLint)** pour analyser la qualité du code dès la phase de développement.

Contexte

À l'issue de l'Atelier 2, le modèle AutoLoc est complet : neuf entités du package **domain**, reliées par leurs associations. Il manque la couche qui permet de lire et d'écrire ces entités en base. Cet atelier construit la **couche Repository** dans le package **repository** créé à l'Atelier 1. L'organisation en couches du projet « autoloc-api » reste celle de l'Atelier 1 :

Package (sous tn.esprit.autoloc)	Contenu	Atelier
domain	Entités JPA et énumérations.	Ateliers 1 et 2
repository	Interfaces Spring Data JPA, préfixées `I` : `IContratRepository` ...	Atelier 3 (cet atelier)
service	Couche métier.	Atelier 4
web.controller	Contrôleurs REST.	Atelier 5

Le fil rouge est **Contrat ↔ Paiement** : on réalise d'abord les opérations CRUD avec **CrudRepository** sur le Contrat, ce qui permet d'observer aussi la cascade et l'orphanRemoval. On compare ensuite les interfaces Spring Data sur cette même entité, puis on généralise aux neuf

entités. Cet atelier ne comporte pas de tests automatisés : le comportement est observé avec un petit programme de démonstration et les logs SQL.

Prérequis (Atelier 2)

- Projet « autoloc-api » (package racine `tn.esprit.autoloc`) fonctionnel, connecté à MySQL, avec `spring.jpa.hibernate.ddl-auto=update`.
- Neuf entités dans le package `domain`, associations de l'Atelier 2 implémentées et schéma généré cohérent.
- Logs SQL activés
(`spring.jpa.show-sql=true`, `logging.level.org.hibernate.SQL=DEBUG`, configurés à l'Atelier 1).
- IntelliJ IDEA installé (le plugin SonarQube for IDE sera ajouté à l'étape 3).

2. Rappels théoriques — Les interfaces de Spring Data JPA

Du JPA à Spring Data JPA

Élément	Nature	Rôle
JPA (<code>jakarta.persistence</code>)	Spécification	Définit les annotations (<code>@Entity</code> , <code>@OneToMany</code> ...) et l'API <code>EntityManager</code> .
Hibernate	Implémentation de JPA	Génère le SQL, gère le contexte de persistance et le dirty checking.
Spring Data JPA	Abstraction Spring	À partir d'une simple interface, génère à l'exécution l'implémentation (un proxy) : plus de DAO écrit à la main.

La hiérarchie des interfaces

Spring Data propose plusieurs interfaces, de la plus minimale à la plus complète. Le développeur choisit celle dont il a besoin ; chacune ajoute des méthodes à celles de ses parentes.

```

Repository<T, ID>           interface marqueur, aucune méthode
├── CrudRepository<T, ID>   CRUD de base
│   ├── ListCrudRepository<T, ID> idem, collections renvoyées en List
│   └── PagingAndSortingRepository<T, ID> tri et pagination
│       └── ListPagingAndSortingRepository idem, collections renvoyées en List

JpaRepository<T, ID> = ListCrudRepository
                    + ListPagingAndSortingRepository
                    + QueryByExampleExecutor
                    + méthodes propres à JPA (flush, suppression en lot...)
  
```

IMPORTANT : Depuis Spring Data 3, `PagingAndSortingRepository` **n'étend plus** `CrudRepository`. Une interface qui veut le CRUD **et** la pagination doit déclarer les deux, ou étendre `JpaRepository` qui les réunit déjà.

Repository<T, ID> : l'interface marqueur

Elle ne contient aucune méthode : elle sert uniquement à signaler à Spring Data qu'une interface est un repository, avec le type de l'entité **T** et le type de sa clé primaire **ID**. En l'étendant directement, on **choisit une à une** les méthodes exposées, en recopiant leur signature :

```
public interface IClientRepository extends Repository<Client, Long> {
    Client save(Client client);
    Optional<Client> findById(Long id);
}
```

C'est utile pour restreindre volontairement l'accès (un repository en lecture seule, par exemple). Dans AutoLoc, **on préfère les interfaces complètes ci-dessous**.

CrudRepository<T, ID> : le CRUD de base

Méthode	Rôle	Retour
<code>save(S entity)</code>	Crée ou met à jour une entité (voir tableau ci-dessous).	L'entité enregistrée
<code>saveAll(Iterable<S>)</code>	Enregistre plusieurs entités.	<code>Iterable<S></code>
<code>findById(ID id)</code>	Lit une entité par sa clé primaire.	<code>Optional<T></code>
<code>existsById(ID id)</code>	Teste l'existence d'une ligne.	<code>boolean</code>
<code>findAll()</code>	Lit toutes les entités.	<code>Iterable<T></code>
<code>findAllById(Iterable<ID>)</code>	Lit plusieurs entités à partir de leurs identifiants.	<code>Iterable<T></code>
<code>count()</code>	Compte les entités.	<code>long</code>
<code>deleteById(ID id)</code>	Supprime par identifiant. Aucune exception si l'id n'existe pas (Spring Data 3).	<code>void</code>
<code>delete(T entity)</code>	Supprime une entité.	<code>void</code>
<code>deleteAllById(Iterable<ID>), deleteAll(Iterable<T>), deleteAll()</code>	Suppression de plusieurs entités, ou de toutes les lignes de la table.	<code>void</code>

Les suppressions de `CrudRepository` passent par le contexte de persistance : **la cascade REMOVE et l'orphanRemoval s'appliquent**. Le comportement de **save** dépend de l'état de l'entité :

Situation	Opération déclenchée
Identifiant <code>null</code> (entité nouvelle)	persist : <code>INSERT</code> . L'identifiant est attribué par MySQL (<code>IDENTITY</code>) et renseigné dans l'objet retourné.
Identifiant renseigné (entité existante)	merge : <code>SELECT</code> pour charger la ligne, puis <code>UPDATE</code> si des champs ont changé. Utiliser l'objet retourné , qui peut différer de celui passé en paramètre.

ListCrudRepository<T, ID> : le CRUD avec des List

Identique à `CrudRepository`, mais les méthodes qui renvoient plusieurs entités (`findAll`, `findAllById`, `saveAll`) retournent une `List` au lieu d'un `Iterable`. Plus pratique à manipuler (`size()`, `get(i)`, `stream()`), c'est la variante recommandée dès que l'on veut une liste.

PagingAndSortingRepository<T, ID> : tri et pagination

Méthode	Rôle	Retour
<code>findAll(Sort sort)</code>	Toutes les entités, triées.	<code>Iterable<T></code>
<code>findAll(Pageable pageable)</code>	Une page d'entités, avec tri éventuel.	<code>Page<T></code>

Objet	Rôle	Exemple
<code>Sort</code>	Décrit le tri. Reçoit le nom de l' attribut Java , pas celui de la colonne.	<code>Sort.by("montantTotal").descending()</code>
<code>Pageable</code>	Décrit la page demandée : numéro (à partir de 0), taille, tri.	<code>PageRequest.of(0, 2, tri)</code>
<code>Page<T></code>	Le contenu de la page, plus le nombre total d'éléments et de pages. Déclenche une requête <code>count(*)</code> supplémentaire.	<code>getContent()</code> , <code>getTotalElements()</code> , <code>getTotalPages()</code>

`ListPagingAndSortingRepository` est la variante dont `findAll(Sort)` renvoie une `List`.

JpaRepository<T, ID> : l'interface complète pour JPA

Spécifique à JPA, elle cumule les interfaces précédentes (version `List`) et ajoute des méthodes liées au contexte de persistance :

Méthode	Rôle
<code>flush()</code>	Envoie immédiatement à la base les modifications en attente.
<code>saveAndFlush(S entity)</code>	<code>save</code> suivi d'un <code>flush</code> : l'écriture en base est immédiate.
<code>saveAllAndFlush(Iterable<S>)</code>	Idem pour plusieurs entités.
<code>getReferenceById(ID id)</code>	Renvoie un proxy paresseux, sans requête immédiate ; une <code>EntityNotFoundException</code> est levée à l'accès si la ligne n'existe pas. Utile pour renseigner une clé étrangère sans charger l'entité.
<code>deleteAllInBatch()</code> , <code>deleteAllByIdInBatch(Iterable<ID>)</code>	Suppression en une seule requête DELETE .

IMPORTANT : Les suppressions « en lot » (`deleteAllInBatch`, `deleteAllByIdInBatch`) contournent le contexte de persistance : **la cascade et l'orphanRemoval ne s'appliquent pas**. Supprimer ainsi des contrats laisserait des paiements orphelins, ou ferait échouer la requête sur la contrainte de clé étrangère.

Quelle interface choisir ?

Besoin	Interface	Remarque
Exposer seulement quelques méthodes choisies	<code>Repository</code>	Contrôle total, signatures à recopier.
CRUD simple, sans souci de type de collection	<code>CrudRepository</code>	<code>findAll</code> renvoie un <code>Iterable</code> .
CRUD avec des listes	<code>ListCrudRepository</code>	Aucune méthode supplémentaire.
Tri et pagination seuls	<code>PagingAndSortingRepository</code>	Pas de CRUD : à combiner avec <code>CrudRepository</code> .
Application JPA complète	<code>JpaRepository</code>	Retenue pour AutoLoc : CRUD, listes, tri, pagination, <code>flush</code> .

Indication : Un repository est une simple interface : Spring Data crée le proxy au démarrage, et l'annotation `@Repository` n'est pas nécessaire. **Convention AutoLoc** : le nom de chaque interface repository commence par `I` (`IVehiculeRepository`, `IContratRepository`...).

3. Étapes guidées — Repositories Contrat et Paiement

Étape 1 — Premier repository avec CrudRepository

Dans le package `tn.esprit.autoloc.repository`, créer l'interface :

```
package tn.esprit.autoloc.repository;

import org.springframework.data.repository.CrudRepository;
import tn.esprit.autoloc.domain.Contrat;

public interface IContratRepository extends CrudRepository<Contrat, Long> {
}
```

1. Relancer l'application : elle démarre sans erreur et sans nouvelle instruction DDL.
2. Repérer dans les logs la ligne indiquant que Spring Data a trouvé un repository (**Found 1 JPA repository interface**).
3. Constaté qu'aucune classe d'implémentation n'a été écrite : le proxy est généré à l'exécution.

Étape 2 — Généraliser aux neuf entités

Sur le modèle de `IContratRepository`, créer dans le package `repository` une interface par entité, toutes de type `JpaRepository<Entité, Long>`. Passer aussi `IPaiementRepository` en `JpaRepository`.

Entité (package domain)	Interface (package repository)
Agence	IAgenceRepository
Employe	IEmployeRepository
Vehicule	IVehiculeRepository
Equipement	IEquipementRepository
Client	IClientRepository
Reservation	IReservationRepository
Contrat	IContratRepository (déjà créée)
Paielement	IPaielementRepository
Maintenance	IMaintenanceRepository

```
public interface IVehiculeRepository extends JpaRepository<Vehicule, Long> {
}
```

Relancer l'application : les logs doivent indiquer **Found 9 JPA repository interfaces**.

Indication : Dans une composition, l'enfant n'est normalement modifié qu'à travers son parent. On crée tout de même **IPaielementRepository** pour pouvoir lire les paiements. La règle : **le code métier passe par le Contrat pour créer ou retirer un paiement**.

Étape 3 — Analyser la qualité avec SonarQube for IDE (SonarLint)

SonarQube for IDE (anciennement SonarLint) analyse le code pendant la saisie et signale défauts, mauvaises pratiques et vulnérabilités potentielles.

1. Dans IntelliJ IDEA : ***Settings > Plugins > Marketplace***, rechercher **SonarQube for IDE**, installer puis redémarrer l'IDE.
2. Ouvrir les entités du package **domain** et les interfaces du package **repository** : les problèmes détectés sont soulignés ; ouvrir le panneau SonarQube for IDE pour lire la description et la justification de chaque règle.
3. Lancer une analyse de tout le projet (clic droit sur le projet, ***Analyze with SonarQube for IDE***).
4. Relever **au moins trois anomalies** (imports inutilisés, nom non conforme, méthode ou classe inutile, **@Data** sur une entité...), les corriger et noter pour chacune la règle concernée et la correction apportée.

À retenir : Un outil d'analyse statique ne remplace pas la relecture, mais il détecte très tôt les défauts courants. Les anomalies relevées alimentent la note **docs/repository-notes.md** demandée ci-dessous.

4. Travail à faire — Les autres repositories

1. Vérifier que les neuf interfaces **I...Repository** sont créées dans le package **repository** et étendent **JpaRepository<Entité, Long>**, puis relancer l'application.
2. Committer le travail avec un message clair (ex. « Atelier 3 : couche Repository Spring Data JPA »).

Interface	Étend	Justification
IContratRepository	JpaRepository<Contrat, Long>	CRUD complet, <code>findAll</code> renvoie une <code>List</code> , <code>saveAndFlush</code> disponible.
... (autres interfaces)	à compléter	à compléter

Anomalie SonarQube for IDE	Règle / explication	Correction apportée
à compléter	à compléter	à compléter
à compléter	à compléter	à compléter
à compléter	à compléter	à compléter

5. Livrables attendus

- Les neuf interfaces `I...Repository` dans le package `tn.esprit.autoloc.repository`, toutes de type `JpaRepository<Entité, Long>`.
- Note `docs/repository-notes.md` complétée (choix d'interface, anomalies SonarQube for IDE corrigées).
- Réalisation du projet, commit et push des modifications sur le dépôt Git « AutoLoc ».

6. Critères d'évaluation

- Respect de l'organisation en couches de l'Atelier 1 (`domain`, `repository`) et de la convention de nommage `I...Repository`.
- Exactitude de la couche Repository : type d'entité et de clé, interface choisie, application qui démarre (`Found 9 JPA repository interfaces`).
- Compréhension des interfaces Spring Data : rôle de chacune, différence `Iterable` / `List`, limites des suppressions en lot.
- Maîtrise du CRUD : usage correct de `save`, `Optional`, `deleteById`, et interprétation des logs SQL.
- Exploitation de SonarQube for IDE : anomalies pertinentes relevées et corrigées.
- Respect des conventions Clean Code (Lombok ciblé sans `@Data`, absence de code mort).